

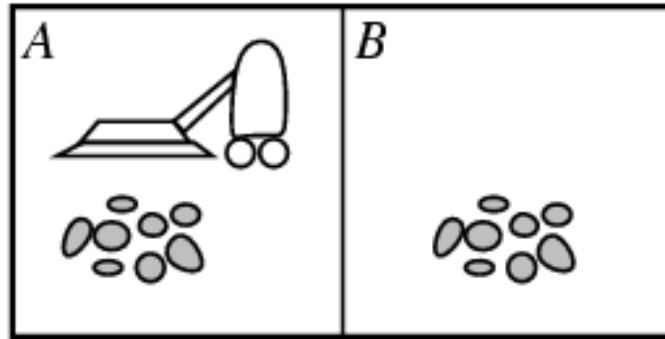
Artificial Intelligence

Chapter 3: Solving Problems by Searching

Adapted from slides available in Russell & Norvig's textbook webpage

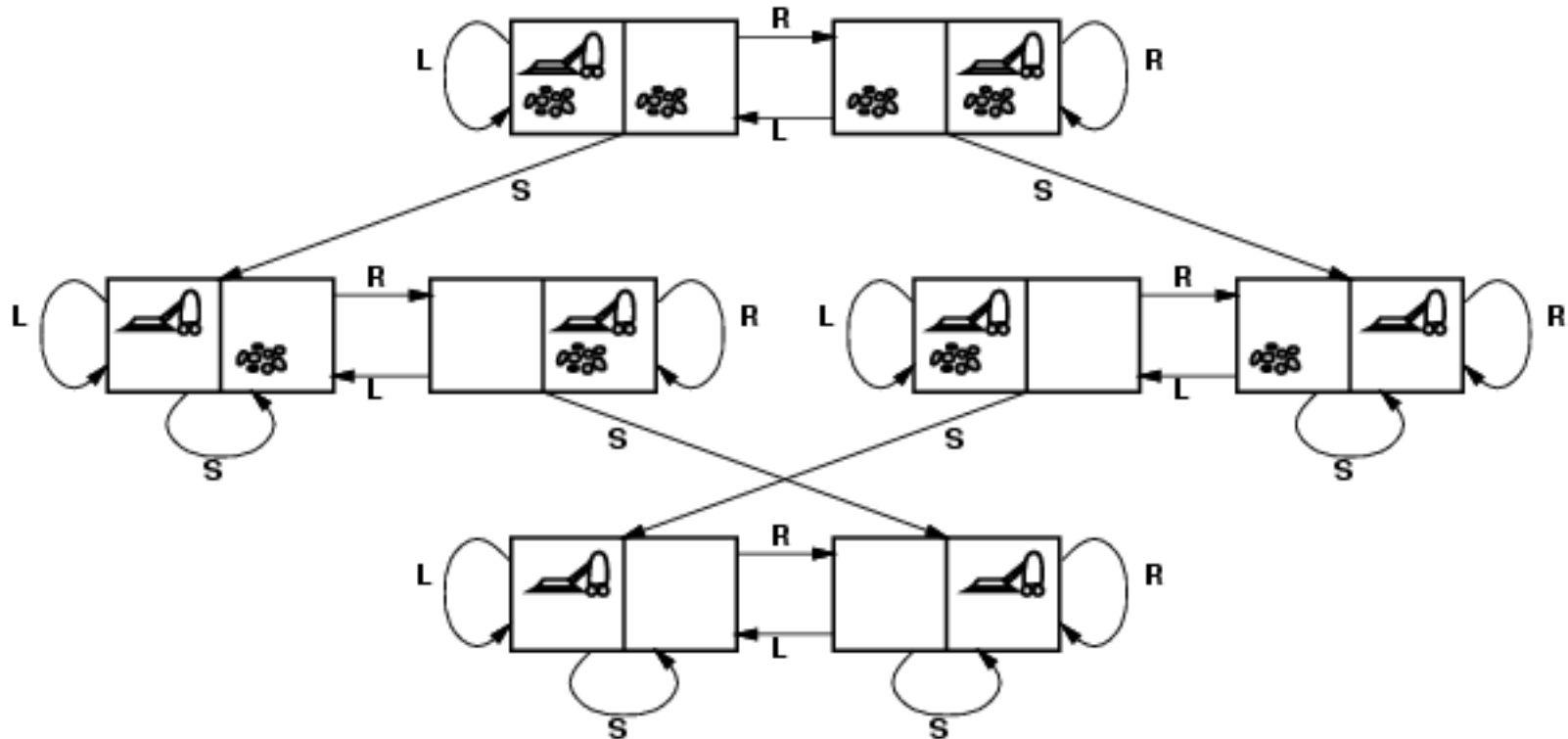


Remember the Vacuum-cleaner world?



- **Percepts:** location and contents, e.g., [A, Dirty]
- **Actions:** *Left*, *Right*, *Suck*

Vacuum world state space graph

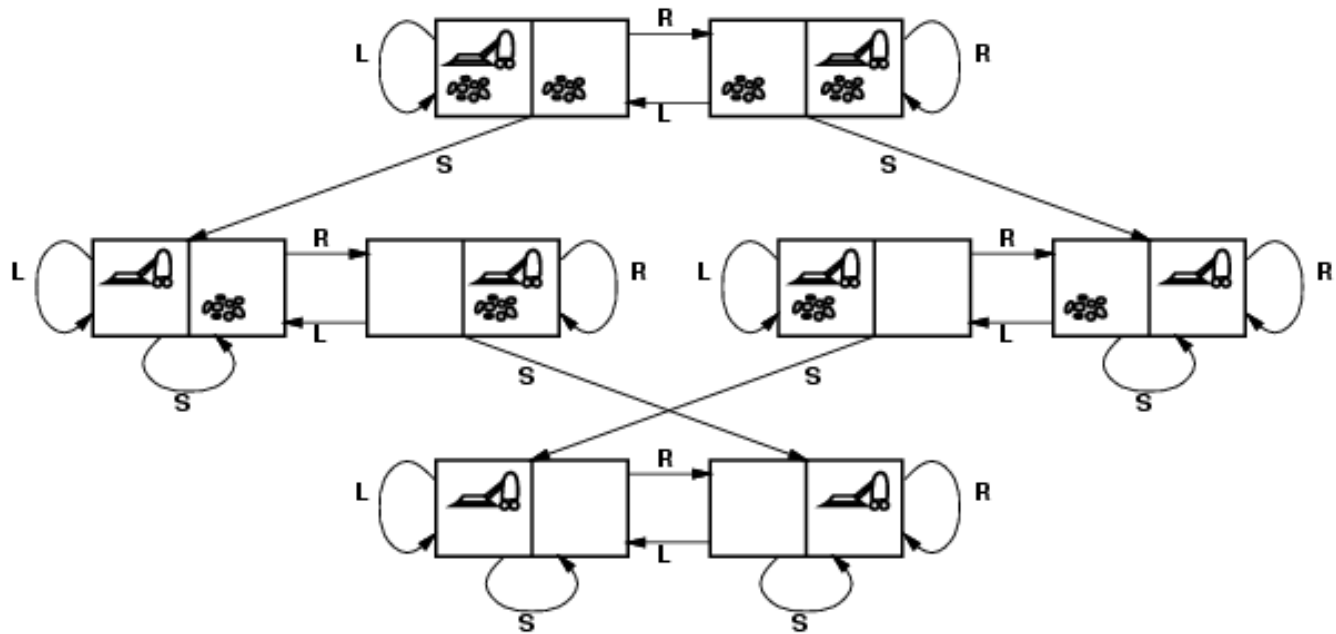


State space: Set of all reachable states. In state space graph,
nodes/vertices = states, links/edges = actions

Formulation of a Problem (**SATGP**)

- A **Problem** is defined by the following items:
 - Set of **States** the agent can be in, with a designated **initial state**
 - Set of **Actions** available to the agent
 - **Transition model** describing what each action does (maps a $\langle \text{state}, \text{action} \rangle$ pair to a state)
 - **Goal test** which determines if a given state is a goal state
 - A **Path cost** function that assigns a numeric cost to each path

Vacuum world state space graph



- states? binary dirt and robot location. Any state can be initial state
- actions? *Left, Right, Suck*
- Transition model? As seen in the state space graph
- goal test? no dirt at all locations
- path cost? 1 per action

Example: The 8-puzzle

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- states?
- actions?
- goal test?
- path cost?

Example: The 8-puzzle

7	2	4
5		6
8	3	1

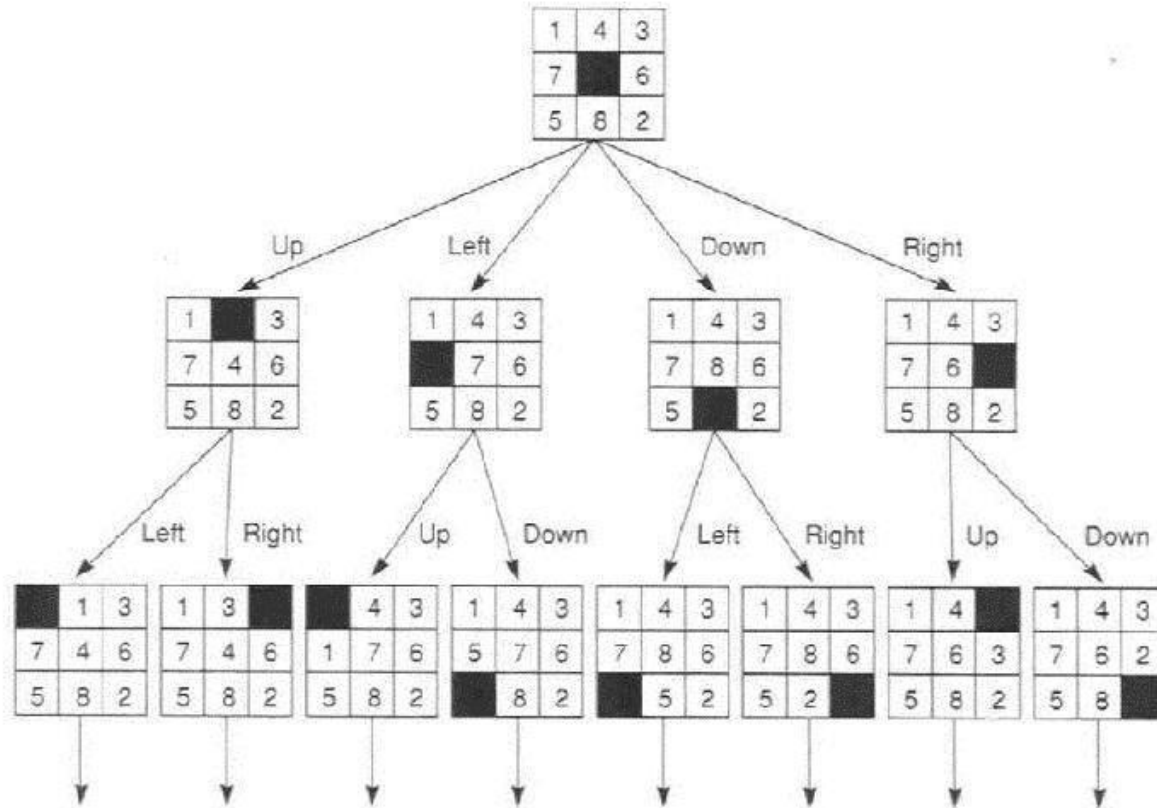
Start State

	1	2
3	4	5
6	7	8

Goal State

- states? locations of tiles
- actions? move blank left, right, up, down
- goal test? = goal state (given)
- path cost? 1 per move

Example: The 8-puzzle



Partial state space graph

Problem

Give a complete problem formulation for each of the following. Choose a formulation that is precise enough to be implemented.

Using only four colors, you have to color a planar map so that no two adjacent regions have the same color.

Initial state: No regions colored.

Actions: Assign a color to an uncolored region.

Transition model: The previously uncolored region has the assigned color.

Goal test: All regions colored, and no two adjacent regions have the same color.

Cost function: Number of assignments.

Problem

Give a complete problem formulation for each of the following. Choose a formulation that is precise enough to be implemented.

A 3-foot tall monkey is in a room where some bananas are suspended from the 8-foot ceiling. He would like to get the bananas. The room contains two stackable, movable, climbable 3-foothigh crates

Initial state: As described in the text.

Actions/Transition model: Hop on crate; Hop off crate; Push crate from one spot to another; Stack one crate on another; Walk from one spot to another; Grab bananas (if standing on crate).

Goal test: Monkey has bananas.

Cost function: Number of actions.

Search strategies

- A search strategy is defined by picking the **order of node expansion**
- Strategies are evaluated along the following dimensions:
 - **completeness**: does it always find a solution if one exists?
 - **time complexity**: number of nodes generated
 - **space complexity**: maximum number of nodes in memory
 - **optimality**: does it always find a least-cost solution?
- Time and space complexity are measured in terms of
 - b : maximum branching factor of the search tree
 - d : depth of the least-cost solution
 - m : maximum depth of the state space (may be ∞)

Uninformed search strategies

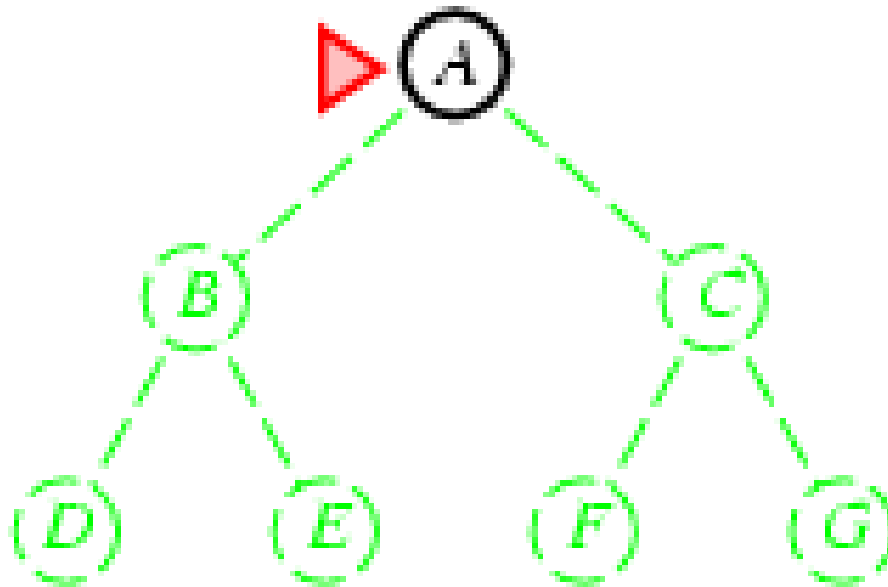
- Uninformed search strategies use only the information available in the problem definition
 - Breadth-first search
 - Uniform-cost search
 - Depth-first search
 - Depth-limited search
 - Iterative deepening search

Basic concept

- **Frontier** (or fringe): The set of all leaf nodes available for expansion at any given point
- The basics of each algorithm:
 - Start from initial node
 - Expand adjacent nodes and put them in the frontier
 - Choose the next node from the frontier for expansion
 - Repeat until goal is found, or some ending criteria is met
- The algorithms differ in the way they choose the next node from the frontier

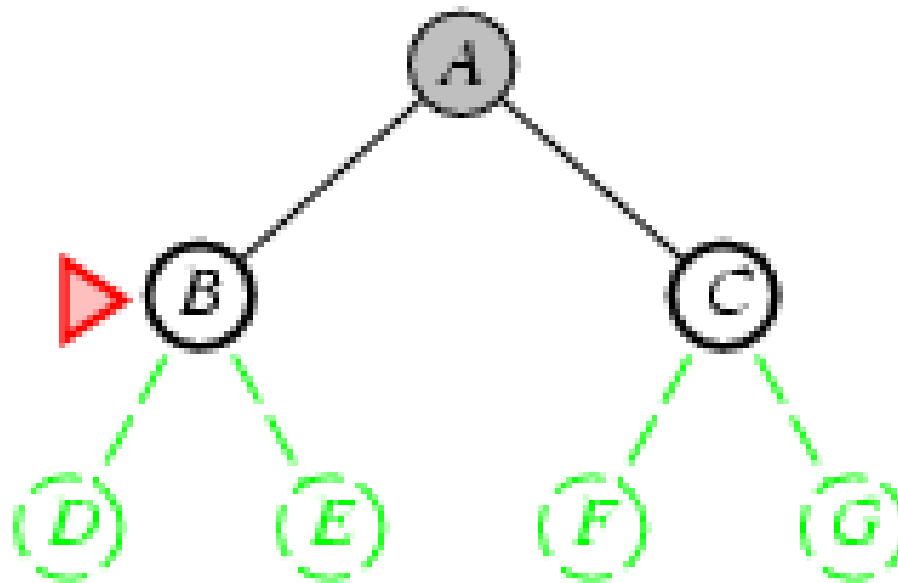
Breadth-first search

- Expand shallowest unexpanded node
- Implementation:
 - *frontier* is a FIFO queue, i.e., new successors go at end



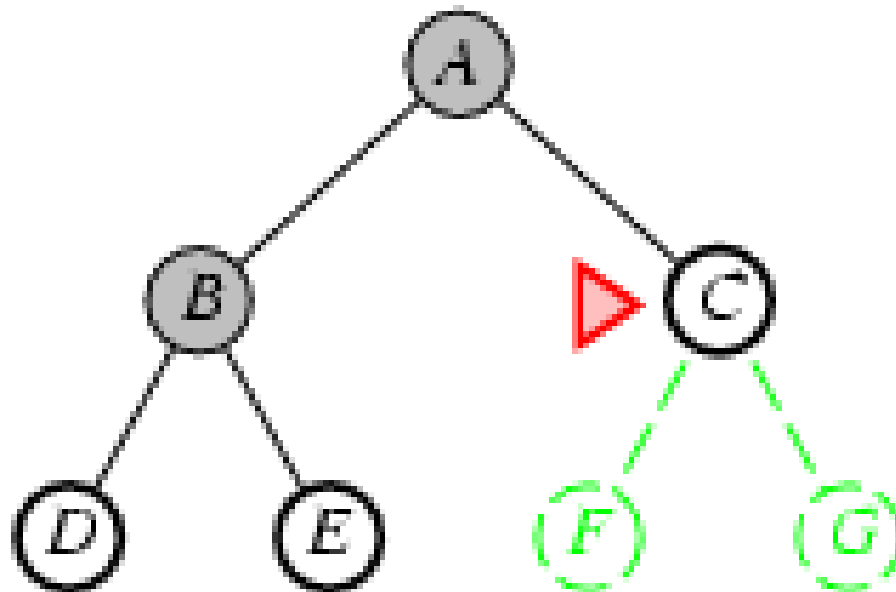
Breadth-first search

- Expand shallowest unexpanded node
- **Implementation:**
 - *frontier* is a FIFO queue, i.e., new successors go at end



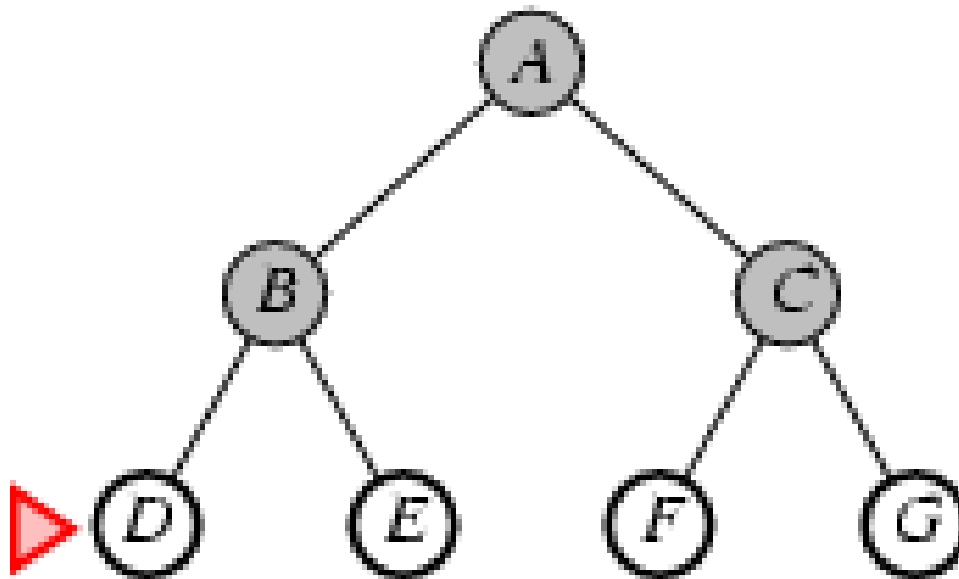
Breadth-first search

- Expand shallowest unexpanded node
- **Implementation:**
 - *frontier* is a FIFO queue, i.e., new successors go at end



Breadth-first search

- Expand shallowest unexpanded node
- **Implementation:**
 - *frontier* is a FIFO queue, i.e., new successors go at end



Properties of breadth-first search

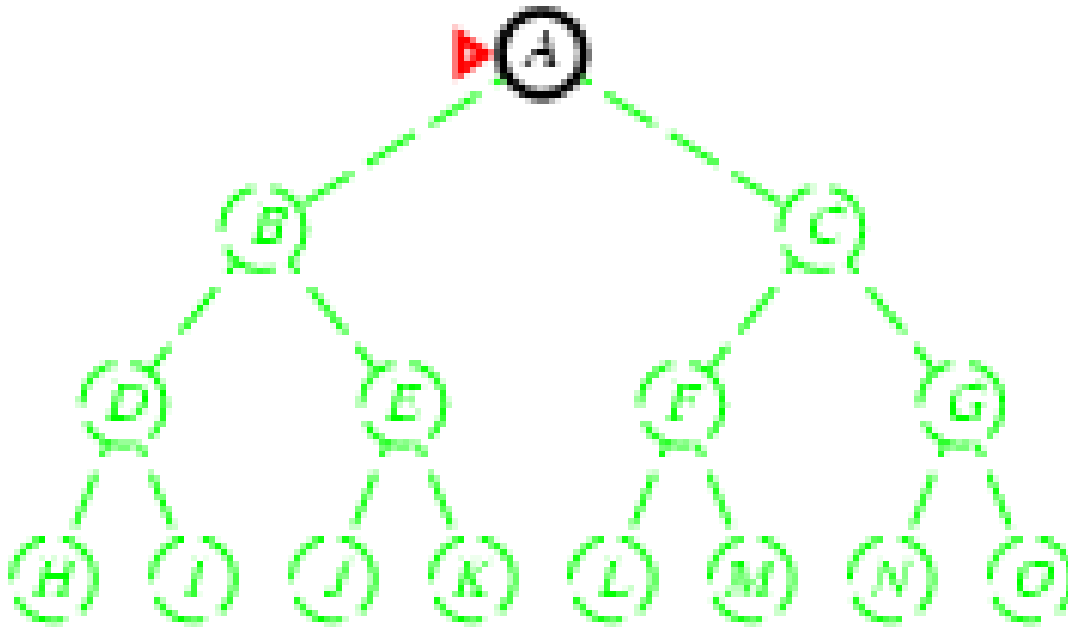
- Complete? Yes (if b is finite)
- Time? $1 + b + b^2 + b^3 + \dots + b^d = O(b^d)$
- Space? $O(b^d)$ (keeps every node in memory)
- Optimal? Yes (if cost = 1 per step)
- Space is the bigger problem (more than time)

Uniform-cost search

- Expand least-cost unexpanded node
- **Implementation:**
 - *frontier* = queue ordered by path cost
- Equivalent to breadth-first if step costs all equal
- **Complete?** Yes, if step cost $\geq \epsilon$
- **Time?** # of nodes with $g \leq$ cost of optimal solution, $O(b^{\lceil C^* / \epsilon \rceil})$ where C^* is the cost of the optimal solution
- **Space?** # of nodes with $g \leq$ cost of optimal solution, $O(b^{\lceil C^* / \epsilon \rceil})$
- **Optimal?** Yes – nodes expanded in increasing order of $g(n)$

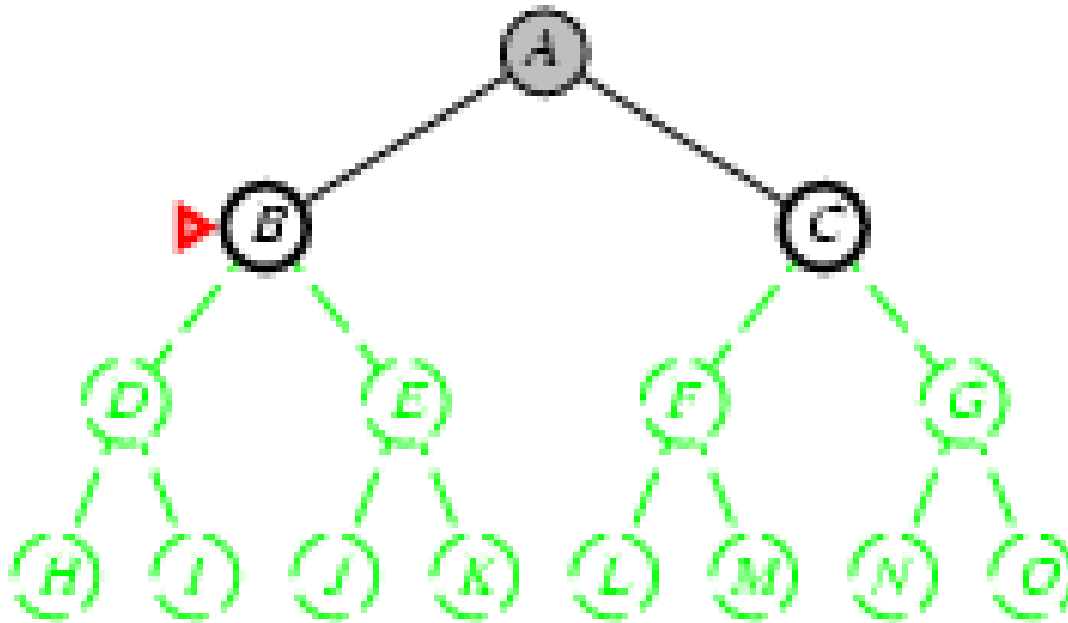
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *frontier* = LIFO stack, i.e., put successors at front



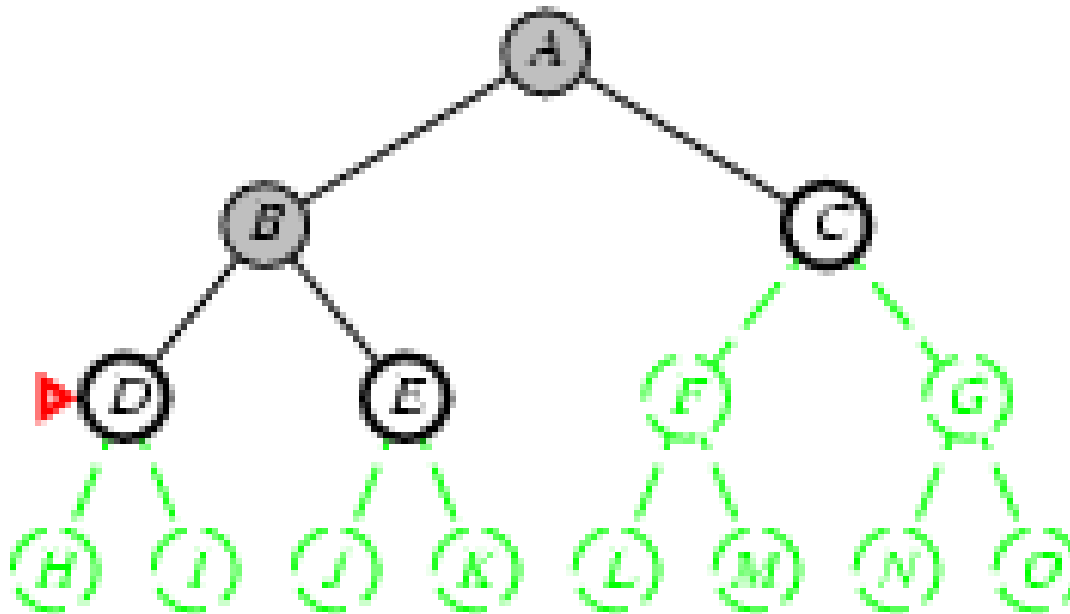
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *frontier* = LIFO stack, i.e., put successors at front



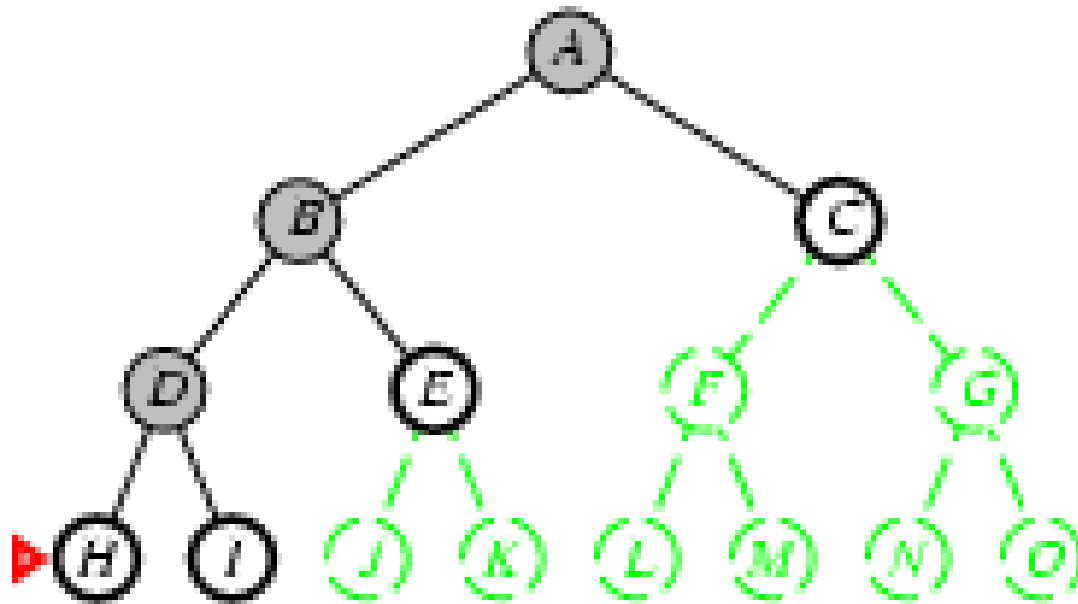
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *frontier* = LIFO stack, i.e., put successors at front



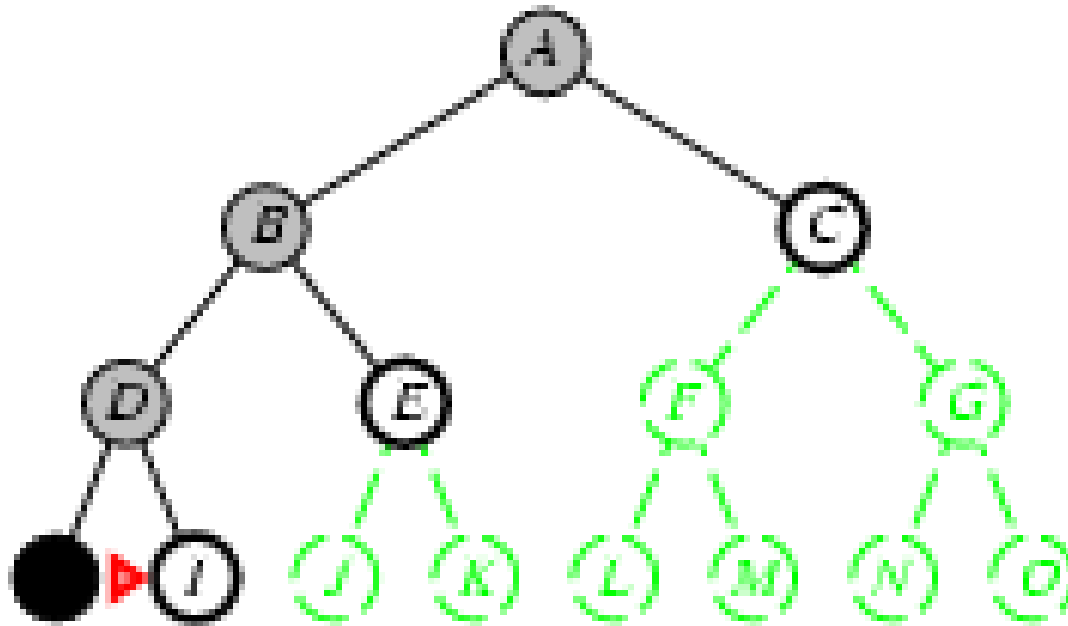
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *frontier* = LIFO stack, i.e., put successors at front



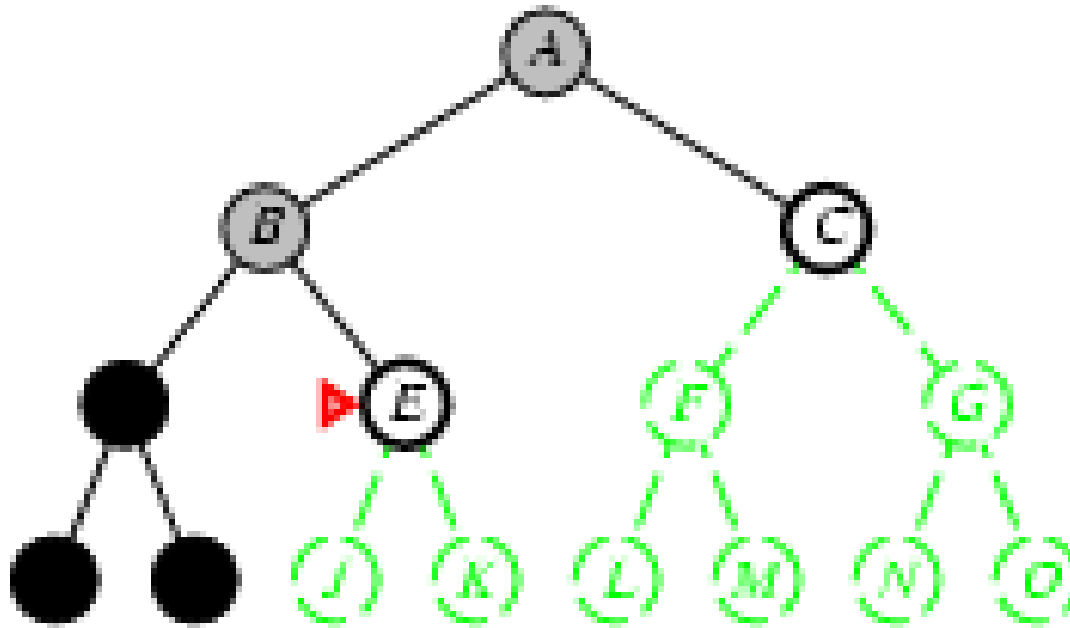
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *frontier* = LIFO stack, i.e., put successors at front



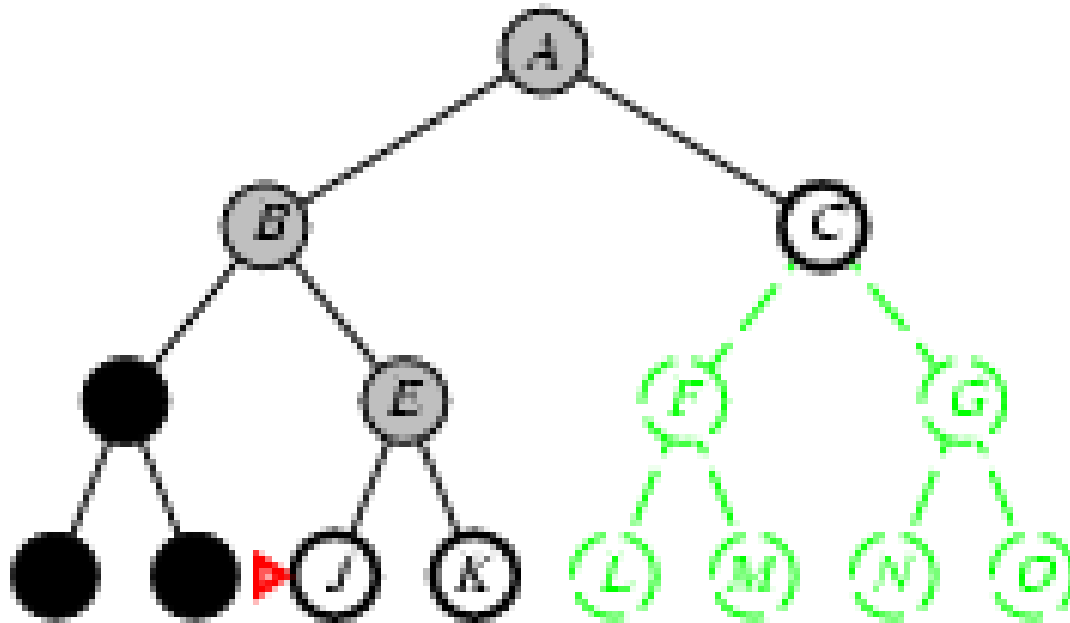
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *frontier* = LIFO stack, i.e., put successors at front



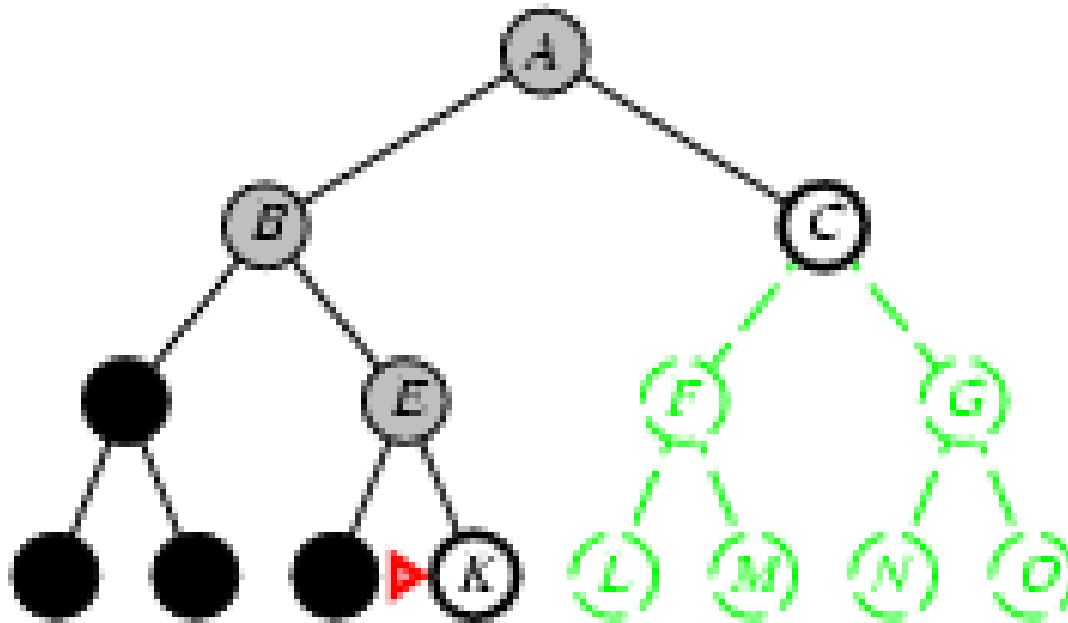
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *frontier* = LIFO stack, i.e., put successors at front



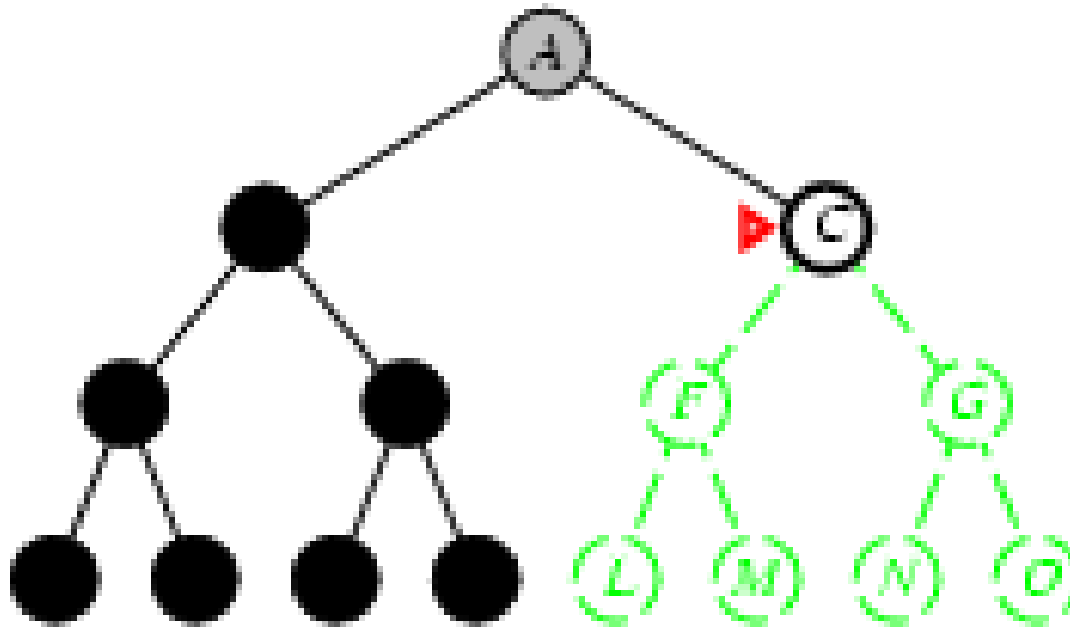
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *frontier* = LIFO stack, i.e., put successors at front



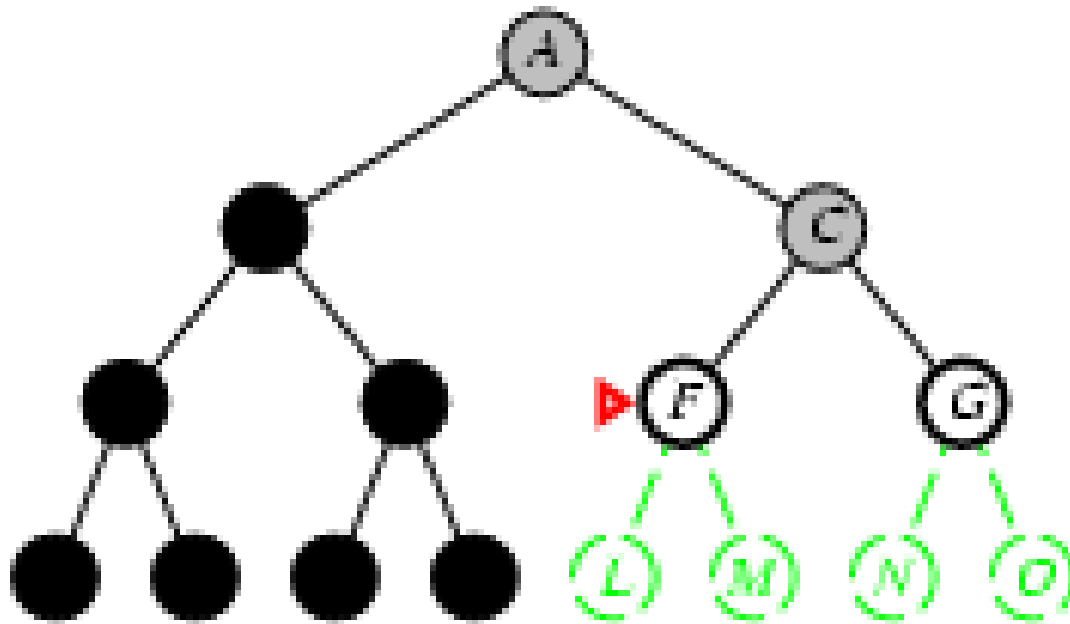
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *frontier* = LIFO stack, i.e., put successors at front



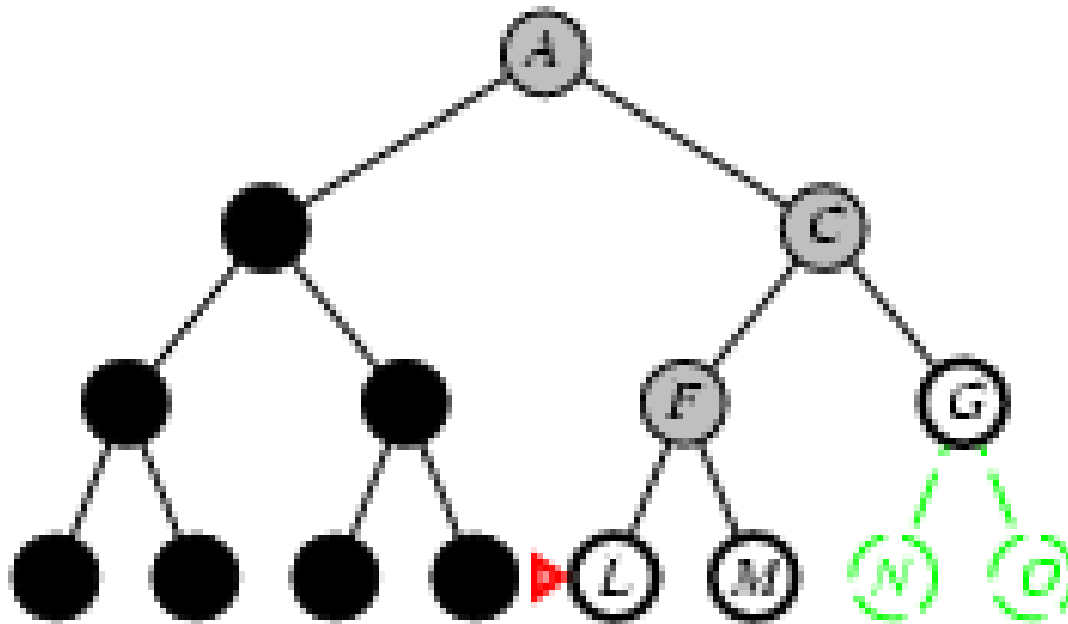
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *frontier* = LIFO stack, i.e., put successors at front



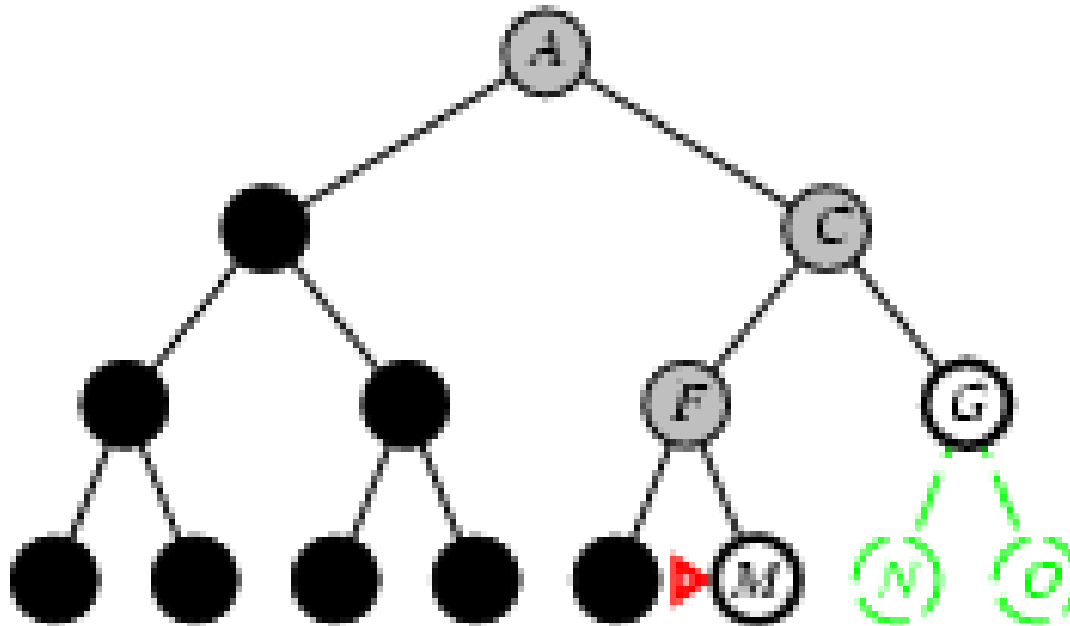
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *frontier* = LIFO stack, i.e., put successors at front



Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *frontier* = LIFO stack, i.e., put successors at front



Properties of depth-first search

- Complete? No: fails in infinite-depth spaces, spaces with loops
 - Modify to avoid repeated states along path
 - complete in finite spaces
- Time? $O(b^m)$: terrible if m is much larger than d
 - but if solutions are dense, may be much faster than breadth-first
- Space? $O(bm)$, i.e., linear space!
- Optimal? No

Depth-limited search

= depth-first search with depth limit l , i.e., nodes at depth l have no successors

- Complete? No
- Time? $O(b^l)$
- Space? $O(bl)$
- Optimal? No

Iterative deepening search $l=0$

Limit = 0



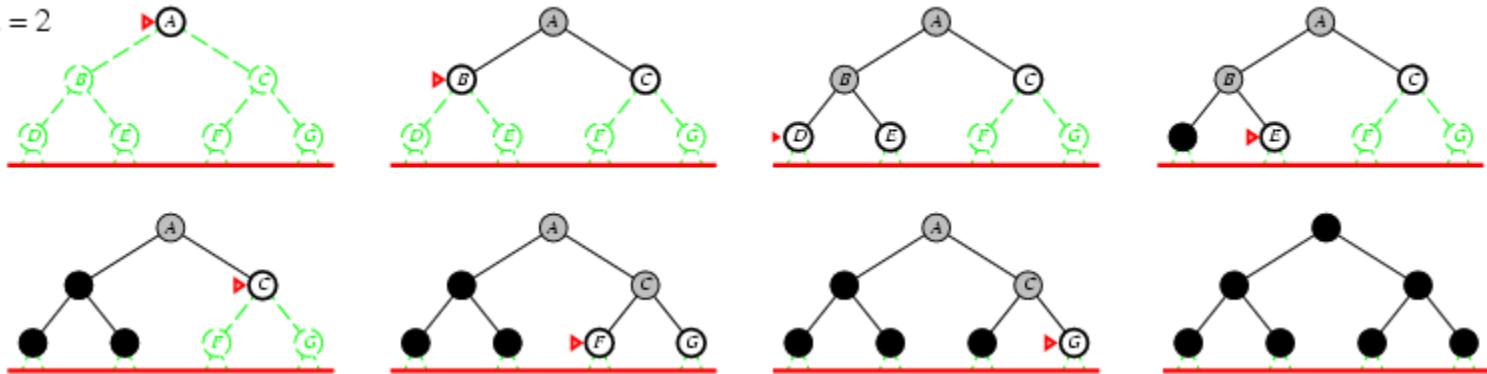
Iterative deepening search $l=1$

Limit = 1



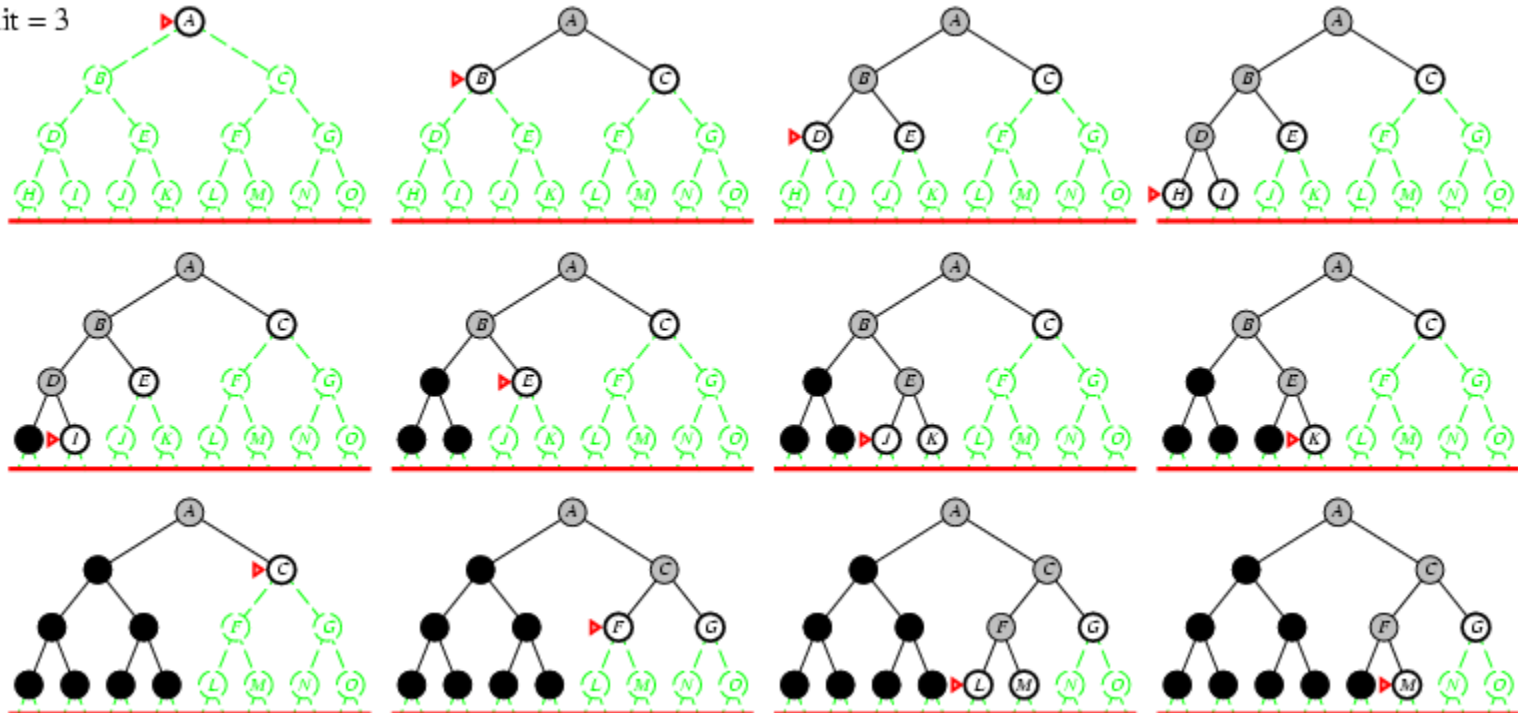
Iterative deepening search $l=2$

Limit = 2



Iterative deepening search $l=3$

Limit = 3



Properties of iterative deepening search

- Complete? Yes
- Time? $(d+1)b^0 + d b^1 + (d-1)b^2 + \dots + b^d = O(b^d)$
- Space? $O(bd)$
- Optimal? Yes, if step cost = 1

Summary of algorithms

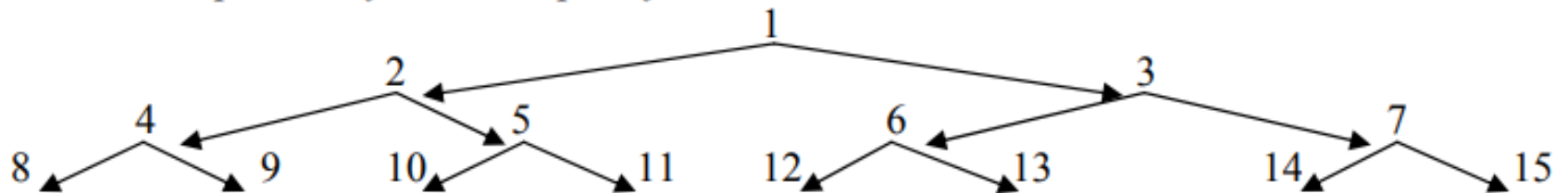
Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes ^a	Yes ^{a,b}	No	No	Yes ^a
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$
Optimal?	Yes ^c	Yes	No	No	Yes ^c

Problem

Consider a state space where the start state is the number 1 and each state k has two successors: numbers $2k$ and $2k+1$.

- a. Draw the portion of the state space for states 1 to 15.
- b. Suppose the goal state is 11. List the order in which nodes will be visited for breadth-first search, depth-limited search with limit 3, and iterative deepening search.

a. Draw the portion of the state space for states 1 to 15.



b. Suppose the goal state is 11. List the order in which nodes will be visited for breadth-first search, depth-limited search with limit 3, and iterative deepening search.

b. Breadth-first: 1 2 3 4 5 6 7 8 9 10 11

Depth-limited: 1 2 4 8 9 5 10 11

Iterative deepening: 1; 1 2 3; 1 2 4 5 3 6 7; 1 2 4 8 9 5 10 11